

Servicio de Datos en Angular



En esta lección veremos cómo agregar el servicio para podernos comunicar con la base de datos de tienda-online definida en Firebase.

Debido a que ahora almacenaremos la información en la base de datos de Firebase, generaremos claves únicas usando Firebase en lugar del valor de id que veníamos trabajando. Para ello necesitamos hacer varios cambios, incluyendo el cambio de id por llave cuando trabajamos con el parámetro de llave única.

La forma en que almacenaremos la información de productos también cambiará, en lugar de usar un arreglo de objetos de tipo Producto, vamos a usar un diccionario en forma de {llave, valor}, donde la llave será generada de manera automática por Firebase, y el valor será el objeto de Producto asociado a la llave. Así quedará almacenada la información en Firebase:



Por ello el objeto de productos ahora será un diccionario de la siguiente forma:

```
// Diccionario con claves únicas generadas por Firebase
productos: { [llave: string]: Producto } = {};
```

Vamos a empezar a realizar los cambios para poder guardar y recuperar ahora la información de Firebase. En primer lugar, creamos un nuevo servicio llamado DatosService:

ng g s datos--skip-tests

Código app.routes.ts (cambiamos el parámetro de :id por :llave):

```
import { Routes } from '@angular/router';
import { ListadoProductosComponent } from './listado-productos/listado-productos.component';
import { FormularioComponent } from './formulario/formulario.component';
import { ErrorComponent } from './error/error.component';

export const routes: Routes = [
  {path: '', component: ListadoProductosComponent},
  {path: 'listado', component: ListadoProductosComponent},
  {path: 'agregar', component: FormularioComponent},
  {path: 'editar/:llave', component: FormularioComponent},
  // Ruta comodín para cualquier otra ruta
  { path: '**', component: ErrorComponent }
];
```

Código producto.model.ts (eliminamos el campo de Id):

```
export class Producto {  
  constructor(  
    public descripcion: string,  
    public precio: number  
  ) {}  
}
```

Esta clase es muy importante, ya que el método guardarProducto es el encargado de agregar un nuevo producto. Con ayuda del método post, firebase de manera automática genera una nueva llave, y es justamente la llave que posteriormente utilizaremos para modificar o eliminar un producto ya existente. Los casos de modificar y eliminar un producto los veremos en la siguiente lección, en esta nos enfocaremos a los casos de listar productos y agregar un nuevo producto.

Código datos.service.ts:

```
import { HttpClient } from '@angular/common/http';  
import { Injectable } from '@angular/core';  
import { Producto } from '../producto/producto.model';  
import { Observable } from 'rxjs';  
  
@Injectable({  
  providedIn: 'root',  
})  
export class DatosService {  
  url = 'https://tienda-online-7f4e5-default-rtdb.firebaseio.com/';  
  
  constructor(private httpClient: HttpClient) {}  
  
  cargarProductos(): Observable<{ [llave: string]: Producto }> {  
    return this.httpClient.get<{ [llave: string]: Producto }>(this.url + 'datos.json');  
  }  
  
  guardarProducto(producto: Producto): Observable<any> {  
    return this.httpClient.post(`${this.url}datos.json`, producto);  
  }  
}
```

Los cambios en la clase de ProductoService van a ser varios, ya que ahora ya no vamos cargarlos desde un arreglo, ni crear cada producto manualmente, así que ya no será necesario el valor de idSiguiente, ni tampoco la función de inicializarProductos. Ahora apoyados del servicio de datos usaremos el método cargarProductos, y esto regresa un observable, al cual nos vamos a suscribir para que cualquier cambio en la base de Firebase

podamos recibir estos cambios y además propagarlos a los demás componentes, como el componente de ListadoProductos, el cual modificaremos posteriormente.

Código producto.service.ts:

```
import { Injectable, EventEmitter } from '@angular/core';
import { Producto } from '../producto/producto.model';
import { Subject } from 'rxjs';
import { DatosService } from '../datos.service';

@Injectable({
  providedIn: 'root',
})
export class ProductoService {
  productos: { [llave: string]: Producto } = {}; // Diccionario con claves únicas
  // generadas por Firebase
  productosActualizados = new Subject<{ [llave: string]: Producto }>(); // Observable
  // para notificar cambios

  constructor(private datosService: DatosService) {}

  obtenerProductos() {
    return this.datosService.cargarProductos();
  }

  setProductos(productos: { [llave: string]: Producto }) {
    this.productos = productos;
    this.productosActualizados.next(this.productos); // Emitimos la lista actualizada
  }

  // Método para agregar o actualizar un producto
  guardarProducto(producto: Producto, llave: string | null = null) {
    if (llave === null) {
      // Caso agregar
      this.datosService.guardarProducto(producto).subscribe(() => {
        this.refrescarProductos(); // Refrescamos la lista después de agregar
      });
    }
  }

  // Método para recargar los productos desde Firebase
  private refrescarProductos() {
    this.obtenerProductos().subscribe(
      (productos: { [llave: string]: Producto }) => {
        this.setProductos(productos); // Actualizamos la lista y notificamos
      }
    );
  }
}
```

```

    );
  }
}

```

Necesitamos hacer algunos cambios más en el componente de producto, y listado de productos, así como el formulario con el objetivo de soportar el diccionario de productos con su llave y valor (Producto).

En la clase de ProductoComponent necesitamos pasar la llave, pero debido a que ya no vamos a utilizar la variable de producto.id, se va a sustituir por el atributo de llave que recibiremos desde el componente padre de ListadoProducto:

Código producto.component.html:

```

<li class="list-group-item text-center">
  {{ producto.descripcion }},
  ${{ producto.precio }}
  <button
    type="submit"
    class="btn btn-info ms-4"
    (click)="editarProducto()">Editar Producto
  </button>
</li>

```

Código producto.component.ts:

```

import { Component, Input } from '@angular/core';
import { Producto } from '../producto.model';
import { Router, RouterModule } from '@angular/router';

@Component({
  selector: 'app-producto',
  standalone: true,
  imports: [RouterModule],
  templateUrl: '../producto.component.html',
  styleUrls: '../producto.component.css'
})
export class ProductoComponent {

  @Input() producto!: Producto;
  @Input() llave!: string; // Recibimos la llave generada por Firebase

  constructor(private router: Router) {}

```

```

editarProducto() {
  // Pasamos el ID en la URL
  this.router.navigate(['/editar', this.llave]);
}
}

```

Desde el componente padre de Listadoproductos vamos a pasar la llave y producto al componente hijo de Producto. Debido a que ya no vamos a recorrer directamente un arreglo de Productos, sino un diccionario en forma de llave, valor, entonces creamos un método llamado obtenerLlaves, el cual recuperará cada llave de cada producto que vamos a iterar y así poder saber cual es el producto que vamos a recuperar o modificar.

Código listado-productos.component.html:

```

<div class="container mt-3">
  <h3 class="text-warning">Listado de Productos</h3>

  <div>
    <button class="btn btn-primary my-5" (click)="agregarProducto()">Agregar
    Producto</button>
  </div>

  <ul class="list-group w-50 mx-auto">
    @for (llave of obtenerLlaves(); track llave) {
      @if (productos[llave]) {
        <app-producto [producto]="productos[llave]" [llave]="llave"></app-producto>
      }
    }
  </ul>
</div>

```

El componente de ListadoProductos es el encargado de actualizar la lista de productos como su nombre lo dice, por lo tanto, nos suscribimos al atributo de productosActualizados del servicio de productos, y con cada cambio actualizaremos la lista de productos. Esto lo haremos con cada cambio, ya sea al momento de cargar la aplicación en el método ngOnInit, o con cualquier otro cambio mandando a llamar al método cargarProductos.

Código listado-productos.component.ts:

```

import { Component, OnInit } from '@angular/core';
import { ProductoComponent } from '../producto/producto.component';
import { ProductoService } from '../producto.service';
import { Router } from '@angular/router';

```

```
import { Subscription } from 'rxjs';
import { Producto } from '../producto/producto.model';

@Component({
  selector: 'app-listado-productos',
  standalone: true,
  imports: [ProductoComponent],
  templateUrl: './listado-productos.component.html',
  styleUrls: ['./listado-productos.component.css'],
})
export class ListadoProductosComponent implements OnInit {
  productos: { [llave: string]: Producto } = {};
  productosSubscription: Subscription | null = null;

  constructor(
    private productoService: ProductoService,
    private router: Router
  ) {}

  ngOnInit(): void {
    this.cargarProductos();

    // Escuchar los cambios en la lista de productos
    this.productosSubscription =
this.productoService.productosActualizados.subscribe((productos) => {
    this.productos = productos;
  });
}

cargarProductos() {
  this.productoService
    .obtenerProductos()
    .subscribe((productos: { [llave: string]: Producto }) => {
      this.productos = productos;
      this.productoService.setProductos(productos);
    });
}

obtenerLlaves(): string[] {
  if (this.productos) {
    return Object.keys(this.productos);
  }
  return [];
}
```

```

agregarProducto() {
  this.router.navigate(['agregar']);
}

ngOnDestroy(): void {
  if(this.productosSubscription !== null)
    this.productosSubscription.unsubscribe();
}
}

```

Finalmente, el componente de formulario, debido a que ya no estamos usando un id, sino una llave, vamos a modificar esta funcionalidad.

Código formulario.component.html:

```

<h5 class="text-success mt-3 text-center">Detalle Producto</h5>

<form class="my-4 w-50 mx-auto" #f="ngForm" (ngSubmit)="f.form.valid &&
guardarProducto()">
  <!-- Primera fila: Inputs de descripción y precio -->
  <div class="d-flex justify-content-between align-items-center">
    <input type="text" id="descripcion" name="descripcion"
      placeholder="Descripción Producto" class="form-control me-2 w-75"
      autocomplete="off" [(ngModel)]="descripcionInput" required />

    <input type="number" id="precio" name="precio"
      placeholder="Precio" class="form-control w-25"
      [(ngModel)]="precioInput" required />
  </div>

  <!-- Segunda fila: Botones de Guardar, Eliminar, Cancelar -->
  <div class="d-flex justify-content-center mt-3">
    <button type="submit" class="btn btn-primary mx-3">Guardar Producto</button>

    <!-- Botón de eliminar solo si hay una llaveProducto -->
    @if (llaveProducto !== null) {
      <button type="button" class="btn btn-danger mx-3" (click)="eliminarProducto()">
        Eliminar Producto
      </button>
    }

    <button type="button" class="btn btn-warning mx-3"
      (click)="cancelar()">Cancelar</button>
  </div>
</form>

```

De momento cierta funcionalidad, como la de modificar o eliminar aún no debemos probarla, solo nos estamos enfocando en la funcionalidad de listar y agregar productos de momento. Por ello, desactivaremos la funcionalidad de modificar o eliminar.

Código formulario.component.ts:

```
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';
import { ProductoService } from '../producto.service';
import { Router, ActivatedRoute } from '@angular/router';
import { Producto } from '../producto/producto.model';

@Component({
  selector: 'app-formulario',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './formulario.component.html',
  styleUrls: ['./formulario.component.css']
})
export class FormularioComponent {

  llaveProducto: string | null = null;
  descripcionInput: string = '';
  precioInput: number | null = null;

  constructor(
    private productoService: ProductoService,
    private router: Router,
    private route: ActivatedRoute
  ) {}

  ngOnInit(): void {
    // Verificamos si estamos en modo edición utilizando snapshot
    const llave = this.route.snapshot.paramMap.get('llave');
    if (llave) {
      const producto = this.productoService.getProductoByLlave(llave);
      if (producto) {
        this.llaveProducto = llave;
        this.descripcionInput = producto.descripcion;
        this.precioInput = producto.precio;
      }
    }
  }
}
```

```

guardarProducto() {
  if (this.descripcionInput.trim() === '' || this.precioInput == null ||
this.precioInput <= 0) {
    console.log('Debe ingresar una descripción y un precio válidos');
    return;
  }
  const producto = new Producto(this.descripcionInput, this.precioInput);
  this.productoService.guardarProducto(producto, this.llaveProducto);
  this.limpiarFormulario();
  this.router.navigate(['/']);
}

cancelar() {
  this.limpiarFormulario();
  this.router.navigate(['/']);
}

eliminarProducto() {
  if (this.llaveProducto !== null) {
    //this.productoService.eliminarProducto(this.productoId); // Llamamos al servicio
para eliminar
    this.limpiarFormulario();
    this.router.navigate(['/']);
  }
}

limpiarFormulario() {
  this.llaveProducto = null;
  this.descripcionInput = '';
  this.precioInput = null;
}
}

```

Finalmente, para que todo funcione correctamente, es necesario registrar el proveedor HttpClient. Debido a que estamos usando componentes standalone, entonces no existe el archivo app.module.ts. Por lo tanto, los servicios los vamos a registrar directamente en el archivo main.ts. El archivo se debe modificar y queda como sigue:

```

import { bootstrapApplication } from '@angular/platform-browser';
import { provideHttpClient } from '@angular/common/http';
import { appConfig } from './app/app.config';
import { AppComponent } from './app/app.component';

bootstrapApplication(AppComponent, {

```

```
...appConfig, // Desestructuramos el appConfig correctamente
providers: [
  provideHttpClient(),
  ...appConfig.providers // Desestructuramos los providers del appConfig
]
}).catch((err) => console.error(err));
```

Para probar estos cambios primero debemos eliminar cualquier información que tengamos en la base de datos, para así poder comenzar a agregar productos desde cero.



Con estos cambios podemos agregar un nuevo producto y esto irá listado los productos que vayamos agregando. Vamos a seguir con los casos de actualizar y eliminar un producto.

Saludos!

Ing. Ubaldo Acosta

Fundador de [GlobalMentoring.com.mx](http://www.globalmentoring.com.mx)